```
import 'dotenv/config';
import { createApp } from './app.js';
import { ConfigError, loadConfig } from './config.js';
import { createConversations, type RenderPlan } from './handler.js';
import { planCaption, renderPlanImage } from './services/drawPlan.js';
import { createCoach } from './services/sinucaCoach.js';
import { createWhatsAppClient } from './services/whatsapp.js';
import { createZapiClient } from './services/zapi.js';
import { createTwilioClient } from './services/twilio.js';
import { createPaywall, createHotmartPaywall, createMemoryStore, createPgStore, l
import express from 'express';
import { createVoice } from './services/voice.js';
import { errorInfo, logger } from './utils/logger.js';

let config;
try {
  config = loadConfig();
} catch (err) {
  if (err instanceof ConfigError) {
    console.error(`[config] ${err.message}`);
    process.exit(1);
  }
  throw err;
}

if (config.whatsappEnabled && config.skipSignatureVerification) {
  logger.warn('SKIP_SIGNATURE_VERIFICATION=true: assinatura do webhook NÃO será v
}

const coach = createCoach({ apiKey: config.openaiApiKey, model: config.openaiMode
const conversations = createConversations();
const render: RenderPlan = (d, photo, mime) => {
  const png = renderPlanImage(d, photo, mime);
  return png ? { png, caption: planCaption(d) } : null;
};

const whatsapp = config.whatsappEnabled
  ? createWhatsAppClient({ token: config.whatsappToken, phoneNumberId: config.pho
  : undefined;
const zapiClient = config.zapiEnabled
  ? createZapiClient({ instanceId: config.zapiInstanceId, token: config.zapiToken
  : undefined;
const voice = config.voice ? createVoice(config.voice) : undefined;
```

```
let hotmartConfig;
try {
  hotmartConfig = loadHotmartConfig();
} catch (err) {
  console.error(`[config] ${(err as Error).message}`);
  process.exit(1);
}
let paywall = config.asaas ? createPaywall(config.asaas) : undefined;
let dbOk = false;
if (hotmartConfig) {
  let store = hotmartConfig.databaseUrl ? createPgStore(hotmartConfig.databaseUrl
  try {
    await store.init();
    dbOk = store.persistent;
  } catch (err) {
    logger.error('Não consegui abrir o banco: usando memória (quem pagou some se
    store = createMemoryStore();
  }
  if (!store.persistent) logger.warn('Hotmart SEM banco (DATABASE_URL): quem pago
  paywall = createHotmartPaywall({ ...hotmartConfig, store });
}
// O aviso de "pagamento confirmado" sai pela Z-API se ela estiver ligada; senão
const notifier = zapiClient ?? whatsapp;

const hotmartPaywall = hotmartConfig ? paywall : undefined;
const inner = createApp({
  webhook: whatsapp
    ? {
        verifyToken: config.verifyToken,
        appSecret: config.appSecret,
        skipSignatureVerification: config.skipSignatureVerification,
        allowedNumbers: config.allowedNumbers,
        onMessage: (msg) => conversations.handle(msg, { whatsapp, coach, render,
      }
    : undefined,
  demo: config.demoCode ? { code: config.demoCode, coach } : undefined,
  zapi: zapiClient
    ? {
        secret: config.zapiWebhookSecret,
        allowedNumbers: config.allowedNumbers,
        onMessage: (msg) => conversations.handle(msg, { whatsapp: zapiClient, coa
      }
    : undefined,
  asaas:
    !hotmartConfig && paywall && config.asaas && notifier
      ? { token: config.asaas.webhookToken, paywall, notify: (phone, text) => not
```

```
      : undefined,
  twilio: config.twilioEnabled
    ? (() => {
        const twilioClient = createTwilioClient({ accountSid: config.twilioAccoun
        return {
          authToken: config.twilioAuthToken,
          publicUrl: config.twilioPublicUrl,
          allowedNumbers: config.allowedNumbers,
          client: twilioClient,
          handle: (msg, wa) => conversations.handle(msg, { whatsapp: wa, coach })
        };
      })()
    : undefined,
});

// Webhook da Hotmart fica na frente do resto do app.
const app = express();
app.disable('x-powered-by');
if (hotmartConfig && hotmartPaywall && notifier) {
  app.use(createHotmartRouter({ hottok: hotmartConfig.hottok, paywall: hotmartPay
}
app.use(inner);

const server = app.listen(config.port, () => {
  logger.info('Sinuca AI no ar', {
    port: config.port,
    model: config.openaiModel,
    whatsapp: config.whatsappEnabled,
    demo: Boolean(config.demoCode),
    zapi: config.zapiEnabled,
    twilio: config.twilioEnabled,
    voz: Boolean(config.voice),
    cobranca: hotmartConfig ? 'hotmart' : config.asaas ? (config.asaas.sandbox ?
    banco: dbOk,
    graphApiVersion: config.graphApiVersion,
    knowledgeLoaded: config.knowledge.length > 0,
    allowedNumbers: config.allowedNumbers.length,
  });
});

function shutdown(signal: string) {
  logger.info('Encerrando servidor', { signal });
  server.close(() => process.exit(0));
  setTimeout(() => process.exit(0), 10_000).unref();
}
process.on('SIGTERM', () => shutdown('SIGTERM'));
```

```
process.on('SIGINT', () => shutdown('SIGINT'));
process.on('unhandledRejection', (err) => logger.error('Promise rejeitada sem tra
```